SET C

BATCH 2

SRM Institute of Science and Technology
College of Engineering and Technology
School of Computing
SRM Nagar, Kattankulathur – 603203, Chengalpattu District, Tamilnadu
Academic Year: 2025-26 (ODD)

Test: FJ - II **Date: 26/09/2025**
Course Code & Title: 21CSC201J Data structures and Algorithm **Duration:** 1 hour 40 min
Year & Sem: II Year / III Sem **Max. Marks:** 50

Course Articulation Matrix:

Course Outcome	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	Program Specific Outcomes		
													PSO-1	PSO-2	PSO-3
CO1	1	2	3	-	-	-	-	-	-	-	-	3	3	-	-
CO2	2	3	3	-	-	-	-	-	-	-	-	3	3	-	-
CO3	2	3	3	-	-	-	-	-	-	-	-	3	3	-	-
CO4	2	3	3	-	-	-	-	-	-	-	-	3	3	-	-
CO5	3	2	3	-	-	-	-	-	-	-	-	3	3	-	-

Q. No	Question
1	Consider an array arr = [10, 20, 30, 40, 50] with index starting at 0. What is the output of the following loop? i ← 0 while i < 5 do print(arr[i]) i ← i + arr[i] % 3 end while A. 10 20 30 40 50 B. Infinite Loop C. 10 20 30 50 D. 10 20 30 **Right Answer: 10 20 40 50**
2	In a Josephus problem with n = 16 people standing in a circle and every third person (k=3) is eliminated, what is the position of the last survivor (1-based indexing)? **Position 8 is the last survivor.**
3	Consider a circular queue of capacity 5. Initially, front = -1 and rear = -1. After the following operations: enqueue(10), enqueue(20), dequeue(), enqueue(30), enqueue(40), enqueue(50), what will be the final values of front and rear (assuming 0-indexed array)? **A. front = 1, rear = 4** B. front = 1, rear = 0 C. front = 2, rear = 0 D. front = 2, rear = 1
4	What is the postfix expression of the following infix expression: (A + B * (C ^ D - E) ^ (F + G * H)) – I

Solution : A B C D ^ E - F G H * + ^ * + I -

5 You are given a **sorted** singly linked list with multiple repeated values. Write a Pseudocode to compress the list such that all duplicates are removed

Example: Input: 10 → 10 → 20 → 30 → 30 → 40 → NULL

Output: 10 → 20 → 30 → 40 → NULL

Solution:

```
// Function to remove duplicates from a sorted linked list
struct Node* removeDuplicates(struct Node* head) {
   struct Node* current = head;
   struct Node* temp;

   if (head == NULL)
      return head;

   while (current->next != NULL) {
      if (current->data == current->next->data) // Duplicate found: skip the next node
         {       temp = current->next;
                  current->next = current->next->next;
                  free(temp);   // free memory of removed node
               } else {
                  current = current->next;            // Move forward
               }
          }
          return head;
        }
```

6 Consider the queue given below which has FRONT = 1 and REAR = 5.



i) Perform the following operations on the queue:

(a) Add F

(b) Delete two letters

(c) Add G

(d) Add H

(e) Delete four letters

(f) Add I

A B C D E
F=1, R=5

a) Add F	A B C D E F	(F=1, R=6)
b) Delete two letters	C D E F	(F=3, R=6)
c)Add G	C D E F G	(F=3, R=7)
d) Add H	C D E F G H	(F=3, R=8)
e) Delete four letters	G H	(F=7, R=8)
f) Add I	G H I	(F=7, R=9)

ii) Write pseudocode for Add () and Delete ()

```
Procedure Add(Q, ITEM)
   if REAR = SIZE then
      print "Queue Overflow"
   else
      REAR ← REAR + 1
      Q[REAR] ← ITEM
      if FRONT = 0 then
         FRONT ← 1
      end if
   end if
End Procedure
```

```
Procedure Delete(Q)
   if FRONT = 0 or FRONT >
REAR then
      print "Queue Underflow"
   else
      ITEM ← Q[FRONT]
      FRONT ← FRONT + 1
      if FRONT > REAR then
         FRONT ← 0
         REAR ← 0
      end if
      return ITEM
   end if
End Procedure
```

7 In an operating system, 5 applications (A1, A2, A3, A4, A5) are waiting for CPU time. The scheduler uses Round Robin scheduling with a time quantum of 2 units, and the processes are stored in a circular linked list. The CPU burst times of the applications are: A1 = 5 units, A2 = 3 units, A3 = 6 units, A4 = 4 units and A5 = 2 units respectively. Assume that you have circular linked list of size 5(the nodes contain the CPU burst time of given five applications in the order, node1=A1, node2=A2, node3=A3….).

A) Write the pseudocode to show the execution order of processes cycle by cycle, indicating remaining times after each round**. (6 marks)**

```
void RoundRobin(Process* head, int timeQuantum) {
   Process* ptr = head;
   int allCompleted;

   do {
      allCompleted = 1;  // assume all processes finished
      Process* start = ptr;  // mark start of this cycle

      do {
         if(ptr->remainingTime > 0) {
            allCompleted = 0; // there is at least one process left

            // Execute process for min(timeQuantum, remainingTime)
            if(ptr->remainingTime > timeQuantum) {
               print(ptr->name, "executes for", timeQuantum, "units");
               ptr->remainingTime -= timeQuantum;
            }
```

```
            else {
                        print(ptr->name, "executes for", ptr->remainingTime, "units (finishes)");
                        ptr->remainingTime = 0;
                    }
                    print("Remaining time of", ptr->name, "=", ptr->remainingTime);
                }
                ptr = ptr->next; // move to next process
            } while(ptr != start); // full cycle completed
        } while(!allCompleted); // repeat until all processes finished
    }
```

B)
Remove an application from the list when its execution completes. **(2 marks)**

```
Process* RemoveProcess(Process* head, Process* toRemove) {
    if(head == NULL) return NULL;     // Only one node in the list
    if(head == toRemove && head->next == head) {
        free(toRemove);
        return NULL;
    }
    Process* prev = head;       // Find previous node
    while(prev->next != toRemove)
        prev = prev->next;          // Remove node
    prev->next = toRemove->next;
    if(toRemove == head)        // Update head if needed
        head = toRemove->next;
    free(toRemove);
    return head;
}
```

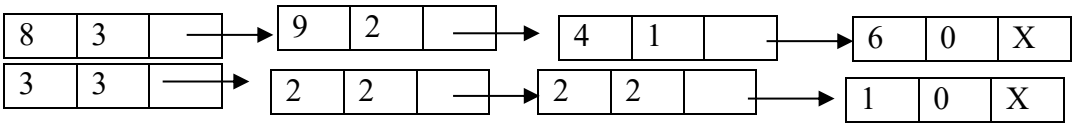
8 Consider that we have an unsorted doubly linked list containing n nodes, write an algorithm to remove duplicate nodes while preserving the original order.

```
1. Initialize pointer current = head
2. While current != NULL do
   2.1 Initialize pointer runner = current->next
   2.2 While runner != NULL do
      2.2.1 If runner->data == current->data then          // Duplicate found, remove runner
         temp = runner
         runner = runner->next                            // Adjust links to remove temp node
         if temp->prev != NULL then
            temp->prev->next = temp->next
         end if
         if temp->next != NULL then
            temp->next->prev = temp->prev
         end if
         free(temp)                                       // delete node
      2.2.2 Else
         runner = runner->next
      End If
   End While
   2.3 Move current = current->next
End While
```

9 Consider the following two polynomials:

- $P_1(x) = 8x^3 + 9x^2 + 4x + 6$
- $P_2(x) = 3x^3 + 2x^2 + 2x + 1$

(a) Illustrate the linked list representation for each of the two polynomials**. (4 Marks)**
(b) Write pseudocode that subtracts two such polynomials and returns the resulting polynomial**.(4 Marks)**




```
PolyNode* SubtractPolynomials(PolyNode* P1, PolyNode* P2) {
   PolyNode* resultHead = NULL;  // head of result list
   PolyNode* resultTail = NULL;
   while(P1 != NULL || P2 != NULL) {
      PolyNode* newNode = (PolyNode*)malloc(sizeof(PolyNode));
      if(P1 != NULL && P2 != NULL) {
         if(P1->exp == P2->exp) {
            newNode->coeff = P1->coeff - P2->coeff;
            newNode->exp = P1->exp;
            P1 = P1->next;
            P2 = P2->next;
         }
         else if(P1->exp > P2->exp) {
            newNode->coeff = P1->coeff;
            newNode->exp = P1->exp;
            P1 = P1->next;
         }
         else { // P2->exp > P1->exp
            newNode->coeff = -P2->coeff; // subtracting => negate
            newNode->exp = P2->exp;
            P2 = P2->next;
         }
      }
      else if(P1 != NULL) { // remaining P1 terms
         newNode->coeff = P1->coeff;
         newNode->exp = P1->exp;
         P1 = P1->next;
      }
      else { // remaining P2 terms
         newNode->coeff = -P2->coeff;
         newNode->exp = P2->exp;
         P2 = P2->next;
      }
      newNode->next = NULL;
      if(resultHead == NULL) {          // Append to result list
         resultHead = resultTail = newNode;
      } else {
         resultTail->next = newNode;
         resultTail = newNode;
      }
   }
   return resultHead;
}
```

10	A popular web browser keeps track of the pages a user visits using a data structure to implement the "Back" functionality. Each time a user visits a new webpage, the page is inserted into the data structure. Clicking the "Back" button deletes the current page from the data structure and navigates to the previous one.

	A user visits the following web pages in this order:
	Home → About → Services → Blog → Contact → Careers
	Then the user clicks the "Back" button three times and then visits a new page **FAQ.**
	1. Identify the Data structure and draw diagrams for every visit of web pages **(1 Mark)**
	2. Write the pseudocode for the "Visit New Page" and "Back" operations. **(3+3 Marks)**
	Explain why the data structure is an appropriate one for implementing the "Back" feature in a web browser. **(1 Mark)**
	
	Answer:
	The **appropriate data structure is a Stack**.
	Home
	Home → About
	Home → About → Services
	Home → About → Services → Blog
	Home → About → Services → Blog → Contact
	Home → About → Services → Blog → Contact → Careers
	
	Home → About → Services → Blog → Contact (Back 1)
	Home → About → Services → Blog (Back 2)
	Home → About → Services (Back 3)
	Home → About → Services →FAQ (Visit FAQ)
	
	`Step 1: IF TOP = MAX-1`
	`PRINT "OVERFLOW"`
	`Goto Step 4`
	`[END OF IF]`
	`Step 2: SET TOP = TOP + 1`
	`Step 3: SET STACK[TOP] = VALUE`
	`Step 4: END`
	
	`Step 1: IF TOP = NULL`
	`PRINT "UNDERFLOW"`
	`Goto Step 4`
	`[END OF IF]`
	`Step 2: SET VAL = STACK[TOP]`
	`Step 3: SET TOP = TOP - 1`
	`Step 4: END`
	
	Stack is perfect for "Back" feature because the most recently visited page is removed first.(LIFO)

11	Convert the following infix expression to its equivalent postfix form using a stack. Infix Expression: A + B * (C ^ D - E) / (F + G ^ H) - I Postfix Expression: ABCD^E-*FGH^+/+I-

Infix symbol	Operator Stack	Postfix
A		A
+	+	A
B	+	AB
*	+*	AB
(	+*(	AB
C	+*(	ABC
^	+*(^	ABC
D	+*(^	ABCD
-	+*(-	ABCD^
E	+*(-	ABCD^E
)	+*	ABCD^E-
/	+/	ABCD^E-*
(	+/(	ABCD^E-*
F	+/(	ABCD^E-*F
+	+/(+	ABCD^E-*F
G	+/(+	ABCD^E-*FG
^	+/(+^	ABCD^E-*FG
H	+/(+^	ABCD^E-*FGH
)	+/	ABCD^E-*FGH^+
-	-	ABCD^E- *FGH^+/+
I	-	ABCD^E- *FGH^+/+I
		ABCD^E- *FGH^+/+I-

Pseudocede (4 Marks)

```
// Function to return precedence of operators
int precedence(char op) {
    if (op == '^') return 3;
    else if (op == '*' || op == '/') return 2;
    else if (op == '+' || op == '-') return 1;
    else return 0;
}
InfixToPostfix(string infix) {   // Function to convert infix to postfix
    Stack  s;          // stack to hold operators
    string postfix = "";    // output expression
    char ch;
    for (i = 0; i < length(infix); i++) {
        ch = infix[i];
        if (ch is operand) {
            // directly add operand to postfix
            postfix = postfix + ch;
        }
        else if (ch == '(') {
            push(s, ch);
        }
        else if (ch == ')') {
            while (!isEmpty(s) && top(s) != '(') {
                postfix = postfix + pop(s);
            }
```

```
            pop(s); // discard '('
        }
        else if (ch is operator) {
            while (!isEmpty(s) && precedence(top(s)) >= precedence(ch)) {
                postfix = postfix + pop(s);
            }
            push(s, ch);
        }
    }
    while (!isEmpty(s)) {       // pop any remaining operators
        postfix = postfix + pop(s);
    }
    return postfix;
}
```

12 In a corporate setting, a centralized print server manages simultaneous print job requests from departments like HR, Finance, and Engineering. Each job has different characteristics: size (pages), urgency (deadline), and importance (departmental priority). Illustrate with an example of five print jobs having different priorities and arrival orders.

- Show how jobs are inserted and processed step-by-step. **(4 Marks)** **Based on the assigned values of deadline and priority draw the diagrams and provide explanation accordingly.**

Pseudocode for Enqueue: (2 Marks)

Insert (HEAD, DATA, PRIORITY):

- **Step 1:** Create new node with DATA and PRIORITY
- **Step 2:** Check if HEAD has lower priority. If true follow Steps 3-4 and end. Else goto Step 5.
- **Step 3:** NEW -> NEXT = HEAD
- **Step 4:** HEAD = NEW
- **Step 5:** Set TEMP to head of the list
- **Step 6:** While TEMP -> NEXT != NULL and TEMP -> NEXT -> PRIORITY > PRIORITY
- **Step 7:** TEMP = TEMP -> NEXT
  [END OF LOOP]
- **Step 8:** NEW -> NEXT = TEMP -> NEXT
- **Step 9:** TEMP -> NEXT = NEW
- **Step 10**: End

Pseudocode for Dequeue: (2 Marks)

Delete(HEAD):

- **Step 1:** Set the head of the list to the next node in the list. HEAD = HEAD -> NEXT.
- **Step 2:** Free the node at the head of the list
- **Step 3:** End